ANTARCTICA

ASSET MANAGEMENT

Antarctica Take-Home: Portfolio Recommendation

Welcome, and thank you for the time you're about to spend on this. We know it's a real cost, and we don't take it lightly.

It's designed to look like a small slice of actual work we do here, not a puzzle with one correct answer.

The scenario

You've been handed last quarter's portfolio for a small internal fund, along with a dataset of daily prices for the assets it holds. Some soft constraints have been communicated by the business. Your job is to recommend revised weights for the next period and present your recommendation in a way that a colleague could pick up and reason about.

We optimise on monthly returns.

What's provided

Four JSON files hosted publicly over HTTPS. Read them directly or download as you prefer:

- **Holdings** — current portfolio, weights, asset identifiers, metadata. `https://antarctica-hiring-data.s3.eu-west-1.amazonaws.com/portfolio-optimisation/2026-04/holdings.json`
- **Prices** — historical daily prices for each asset, tall format. `https://antarctica-hiring-data.s3.eu-west-1.amazonaws.com/portfolio-optimisation/2026-04/prices.json`
- **Benchmark** — daily levels for a reference index. `https://antarctica-hiring-data.s3.eu-east-1.amazonaws.com/portfolio-optimisation/2026-04/benchmark.json`
- **Constraints** — soft constraints the business has asked us to respect. `https://antarctica-hiring-data.s3.eu-west-1.amazonaws.com/portfolio-optimisation/2026-04/constraints.json`

No starter code. No lockfile. No Dockerfile. This is on purpose.

What to deliver

1. **Build a small Next.js application** that reads the data, produces a recommended weight per asset, and displays the result alongside a chart that helps explain the recommendation. Treat this as a production-ready feature that a user could use the next day — it should feel complete, reliable, and usable end-to-end. Build it as you would for a real task at work, not a throwaway exercise.

2. **Deploy your finished app** so we can use it live during the debrief. Share the live URL with your submission.

You're welcome (and encouraged) to commit any AI configuration files that helped you build this — `AGENTS.md`, `CLAUDE.md`, Claude Code skills, Cursor rules, or similar. We'd rather see them than not.

Suggested time

3 to 4 hours. No hard cap. We are not grading on speed, and we'd rather see thoughtful work than hurried completeness.

If you find yourself reaching for the sixth hour, stop and commit what you have. A short note in the repo on where you'd go next is a valid submission.

Submission

1. Create a new GitHub repository for your work. Public or private is up to you. The dataset is fictional, so there's no confidentiality reason for either.
2. Commit as you go. We read commit history, so don't squash everything into a single commit at the end.
3. Before your scheduled debrief, email us the repo URL (and grant read access if it's private).

Everything you need is in this brief and at the public URLs above. You don't need access to any Antarctica-internal systems to complete the assignment.

Debrief and live round

After you submit, we'll schedule a 60-90 minute follow-up session. The first 30 minutes is a debrief where you'll walk us through what you built and answer questions on your decisions and code. The remainder is a live coding round on foundational topics.

What we deliberately haven't told you

We haven't specified the optimisation objective. We haven't told you how to handle missing data, what "minimal UI" means for us, or how much testing to write. Those are decisions we want to see you make, and we'd like to hear the reasoning in your debrief.

Good luck. If anything in the brief is actually unclear (as opposed to intentionally open-ended), email us.